Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

Os objetivos da implementação consistem em montar uma sincronização, com mutex, sem semáforo, o sistema de embarque de um metrô, para que o vagão só saia quando todos os passageiros já tenham embarcado.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Implementação	OK	./sync_car
Relatório	OK	
README + Makefile	OK	

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make
```

Executar (exemplo):

```
./sync_car-runner
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **sync_car.c → arquivo com a lógica**
2. **sync_car.h → arquivo header com structs e protótipos das funções**
3. **sync_car-runner.c → arquivo de teste do professor**
4. **Makefile → instruções de compilação**
5. **README → instruções de uso**
6. **evidências.log → terminal**

Decisões técnicas:

1. **Separar os 3 estados do passageiro em diferentes contadores → com um único contador, não é possível distinguir reserva de confirmação.**
2. **Uso de broadcast, ao invés de signal → não acorda thread que não pode progredir**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Uso de um único contador	Estava usando só um contador, sem distinção (waiting)	Não distingue o reservou de confirmou

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S2	Uso de três contadores	Uso de waiting, boarded, onboarding_pending	
S3			

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Semântica errada do waiting	Erro nos testes	Waiting era tratado como ainda não retornar de wait for car	01-11-2026	Evidência 1
2	S1	Não usei onboarding_pending	Erro nos testes	Não considerava reserva sem confirmação	01-11-2026	Evidência 1
3	S2	Uso de 3 contadores	Sucesso nos testes		04-11-2026	Evidência 2
4						
5						
6						
7						
8						

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.

```
guila@Guilherme:/mnt/c/Users/guila/OneDrive/Documentos/gbl2$ ./sync_car-runner
car entering station with 23 free seats
Error: station_load_car returned early!
```

- Validação final: teste passando / saída correta.

```
guila@Guilherme:/mnt/c/Users/guila/OneDrive/Documentos/gbl2$ ./sync_car-runner
car entering station with 42 free seats
car departed station with 42 new passenger(s) (expected 42)
car entering station with 5 free seats
car departed station with 5 new passenger(s) (expected 5)
car entering station with 46 free seats
car departed station with 46 new passenger(s) (expected 46)
car entering station with 35 free seats
car departed station with 7 new passenger(s) (expected 7)
Looks good!
```

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas): Ajudar a compilar, planejamento, gerar mais testes

Prompts principais (liste, sem história):

1. Quero um passo a passo de implementação, sem código.
2. Quero testes para esse código

O que validei manualmente e como: meus erros

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão: Melhoraria minha gestão de tempo, criaria mais testes e maiores para garantir maior qualidade de código, criaria mais arquivos, ao invés de deixar tudo em um único .c, além de gerar comentários no código para melhorar a leitura de outras pessoas.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Sim
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico.
Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

Tentaria usar casos maiores, com limites maiores, tamanhos variáveis, checaria erros, otimizaria meu código com comentários, de modo a facilitar o entendimento de quem está lendo. Além disso, testaria mais casos, já que, como estive muito atarefado, não me planejei bem e os testes ficaram para o fim..

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.